DATATALKSCLUB

MLZOOMCAMP 2025

Week 3 notes

Carlos Leal León

October 16, 2025

General Index

Figure Index

Equation Index

1 Churn Prediction Introduction

The following steps were described briefly as they are covered later in different sections and practiced with examples in my homework notebook [1].
The notebook followed to learn can be found here [2].
There is an additional notebook that covers *One-Hot Encoding* and use of *StandardScaler* [3].

This project describes as its objective to identify customers that are likely to 'churn' or stop using a service. Where each customer has a score associated with the probability of churning. The company would take appropriate action once they know the score assigned to each customer based on their probability of churning.

1.1 Data preparation

The provided data came from a kaggle dataset [4], it contains 7043 rows (customers) and 21 columns (features).
Steps taken here were:

- Read data: Using pandas read_csv method.
- Look at some observatons: Using pandas head method.
- Identify datatypes: With dtypes attribute of dataframe.

1.2 Validation Framework

The dataset was partitioned in train, validation and test splits using 60%, 20% and 20% respectively. But this time instead of doing each step manually it was done using *train_test_split* from *sklearn.model_selection*.

1.3 Exploratory Data Analysis

Steps here involved:

- Handle missing values
- Look at target variable (Class Imbalance, mean)
- Look at numerical and categorical features

1.4 Feature Importance

Identify which features affect the target variable.

1.4.1 Churn Rate

First the *Churn rate* was reviewed using a few features involving the gender and partner features, simply to know the churn rate of a few of the features.

In this case the global churn is obtained as the mean of the churn in the dataset, and the churn of gender and partner features each using also the mean.

1.4.2 Risk Ratio

Later *Risk ratio* was calculated using the mean and count, this by grouping data using each categorical feature and selecting only the churn feature after grouping to get mean and count. With that data a difference and risk were obtained by subtracting the mean from the global churn and dividing the mean by the global churn respectively. The global churn was obtained by calculating the mean of churn of the entire dataset.

1.4.3 Mutual Information

This refers to How much can you learn about one feature if you know the value of another feature.

Here the *mutual_info_score* [5] of scikit-learn was used.

It was calculated using each of the categorical features and the target feature as parameters of the the mutual_info_score and results were sorted in descending order to identify the features at the top as the ones affecting the target feature more.

1.5 Correlation

It measures the degree of dependency between to variables. Negative if one value increases and other decreases. Positive if both variables increase. The magnitude of the coefficient indicates if the dependency could be low, moderate or strong. It allows measuring the importance of numerical variables. This is only calculated on numerical features.

You can calculate it using *corrwith* [6] and you can get the absolute value if desired.

1.6 One-Hot Encoding (OHE)

As categorical features cannot be measured as they are it isrequired to transform them to a form where this can be done. For this the *DictVectorizer* [7] from scikit-learn was used where you select the numerical and categorical features to create a dictionary, then that dictionary is transformed using the DictVectorizer. This is done for the train adn validation splits in the notebook.

1.7 Logistic Regression

As the target variable has two outcomes churn or no-churn this is a classification task.
The base of this model is the prior *Linear Regression* which outputs a scalar. Now the *sigmoid* [8] function comes to play to turn that number into a probability, describing the possible outcomes using a logistic function from which you can determine wether the probability goes up to 1 or down to 0.
When you visualize the behaviour of the function it looks like an 'S' shaped curve.

1.7.1 Train Logistic Regression

The training of the Logistic Regression has a few details to consider:

- solver: The algorithm to use in the optimization problem. Depending on the scenario you are facing you may have to select a specific solver. The selected solver allows the selection of a specific penalty.
- penalty: The penalty selected indicates the type of Regularization to be applied. In case the solver allows it you can opt to not apply the Regularization using None.
- C (regularization strength): This parameter controls the 'strength' of the Regularization in case the solver allows it and is not None.
- Outcomes: You have negative and positive probability and for what I saw in the notebooks you select the positive probability.

1.7.2 Model Interpretation

To better understand how the model works a smaller set of features was used to train the model, if they are categorical they must be encoded if numerical you can opt to normalize them if required. Once trained you get the intercept of the model of one observation, then you get the coefficients of the same observation and you can review the weighted sum of features manually.

Once using the weighted sum of features you can caculate the sigmoid function and obtain the probability of that observation.

2 Homework Notebook 03

Just mentioning what I read and describe a bit my learning process with this homework.

2.1 Exploratory Data Analysis

Here you deal with missing values using 0.0 for numerical features and 'NA' for categorical features. I simply created a pair of funtions to do each fill for the desired column. Later check there were no more missing values. And review using head.

For the most frequent value in industry feature I did a value_counts with ascending set to False.

2.2 Correlation Matrix

To find the biggest correlation, I used two approaches:

- First approach: Select only numerical features using select_dtypes with int64 and float64 types. Leave target feature out of the selection and with the resulting Dataframe call corr method. As there are not that many features you can look for the biggest correlation searching for it in the Correlation Matrix.
- Second approach: Get the Correlation Matrix again using the numerical features but apply the absolute value to the result. Now pivot column names and sort values in descending order. Once ready apply a filter to select only values that are less than 1.0 and remove duplicate entries. Select the index and value of the first index and that is the biggest correlation (hopefully).

2.3 Validation Framework

You have to split data in train, validation and test partitions with a 60%, 20% and 20% respectively using train_test_split from scikit-learn. Here I found it easier to do a first split with a test size of 0.4 and then do a second split with a test size of 0.5. I did that because first split ends up with train 60% immediately leaving the 40% left to be splitted in half for the validation and test splits.

Later reset the indexes of each split, take the value of each split for y and only remove it from validation and test splits as you have to get the Mutual Information later using the train split including the y or target variable.

2.4 Mutual Information

This step is straightforward you can follow the instructions from the notebook, what is important here is to remember that you are rounding your results and pay attention to the score that each variable has as this helps you determine how much you can learn from the target feature using the available features. So it is possible that a pair of features give almost the same or even the same score in this case.

2.5 Logistic Regression

In my case I have to separate the y vector from the feature matrix and remove the target variable from the feature matrix, that is in the train split.
Now it is time to apply one-hot encoding, in this question (Q4) I used the DictVectorizer as shown in the notebook [2] where you combine the numerical and categorical features, get a dictionary and transform that dictionary using the DictVectorizer. This is done first for the train split to train the Logistic Regression model, the model is trained using liblinear solver, C set to 1.0, max_iter set to 1000 and a random_state of 42. Be aware of the fact that Logistic Regression is regularized bu default unless you change your parameters.

After training the model you have to preprocess the validation split in the same way the train split was preprocessed and use that validation data to make a prediction.

When you look at the predict_proba method output you have to decide which probabilities are the ones you want to review. That is why the slice $[:, 1]$ is applied to the output.

Once you have selected the desired probabilities you can classify each of them by doing a comparation (greater than) with a value of 0.5 to determine if it is a churn or not. Later you can calculate the accuracy of the model by comparing using (equals $==$) between the y of the validation split and the y of the prediction, get the mean and round or not the result (here it requires rounding to two decimals).

2.6 Model Interpretation

The model interpretation is interesting as you realize that you are starting from the Linear Regression base model, but as you can imagine when more features are involved in the process the equation grows a lot, so it is better to take a look at the interpretation using fewer features. In my case I only selected numeric features but simulated doing the entire process as if there were categorical features involved. Then follow the same process as in (Q4) of the previous section and this time you use:

- Model intercept: The intercept of an observation, in my case the first observation.
- Model coefficients: The coefficients calculated of an observation.
- Map feature and values: Here a dictionary is created with help of the zip function to associate the feature name with its corresponding coefficient value.

With this information you can follow the equation by hand and calculate the sum of the bias term with the sum of weights and look how each feature affects the result. The obtained number is then used to calculate the probabilities using the sigmoid function and now you can obtain the predictions, using the prediction of the same observation you used you can see if the result is the same or it approaches to the same value. I saw that some precision was missing but rounding the result to a certain number of decimals helped to compare them.

2.7 Feature Elimination Technique

This sounds like a Brute Force or something like that but what you do is simply train a model with all the features and then remove one by one to compare the accuracy of the original model and subsequent models where one feature is removed from the model. Here I used a copy of the train and validation splits to not affect the original splits and repeat the steps taken to train the Logistic Regression but just before preprocessing the features remove the feature in turn to exclude it from the training process. All of this done in a function.

2.8 Regularized Logistic Regression (Consciously)

I took a look at the distribution of the features and decided to use the *StandardScaler* [9] as show in [3] and also try out the One-Hot Encoding using *OneHotEncoder* [10] instead of *DictVectorizer*.

The steps here are similar to the first trained Logistic Regression but here you change the value of C to manipuate the impact of the Regularization which by the way is set to *Ridge Regression*.

With each model trained I got the prediction on the validation split and calculated the accuracy of the model, store each in a dictionary using the C value as a string key.

References

[1] cleall (github). ”hw notebook” [Online], at: hw-three [Created on October 2025].

[2] DataTalksClub (github). ”03-classification notebook.ipynb” [Online], at: churn notebook [Consulted on October 2025].

[3] DataTalksClub (github). ”03-classification notebook-scaling-ohe.ipynb” [Online], at: scaling-ohe notebook [Consulted on October 2025].

[4] BlastChar. ”Telco Customer Churn” [Online], at: telco-csv [Consulted on October 2025].

[5] scikit learn. ”mutual info score” [Online], at: mutual-info-score [Consulted on October 2025].

[6] pandas. ”corrwith” [Online], at: corrwith [Consulted on October 2025].

[7] scikit learn. ”DictVectorizer” [Online], at: dict-vectorizer [Consulted on October 2025].

[8] SciPy. ”expit” [Online], at: expit [Consulted on October 2025].

[9] scikit learn. ”StandardScaler” [Online], at: standard-scaler [Consulted on October 2025].

[10] scikit learn. ”OneHotEncoder” [Online], at: ohe [Consulted on October 2025].